Part1:

For Figure 1Average HAA Across Channels: What do you see in the average HAA activity? Do responses appear better aligned to stimulus onset or response time? Or both?

In the average HAA, we see that the responses are better aligned at the stimulus for theta waves and better aligned at the response for the high gamma band.The average HAA of the theta response is aligned with the characteristic decrease in HAA after stimulus.

For Figure 3 Histograms at -200ms: Choose one bipolar channel. Do the histograms look normally distributed?

I chose Channel 2, which is the LOF3-LOF2 connection. The histograms do look normally distributed. For both theta and high gamma histograms, the count is normally distributed centered at 0 HAA.

For Figure 4 AUC Curves Across Channels: What do you observe in the data? Could you get good classification with individual bipolar channels? What is the best discrimination that you got (AUC farthest from 0.5)? Which band was it? Was it aligned to response time or stimulus onset?

- **Theta band**: The AUC that is furthest from 0.5 is LOF5-LOF4 (Channel 4) when aligned with the response to the stimulus. We see that at -500ms, the channel has a peak of AUC=0.57.
- **Gamma band:** The AUC that is furthest from 0.5 is LOF3-LOF2 (Channel 2) when aligned with the response to the stimulus onset. At 12 ms, the AUC peaks at 0.7

For Figure 5 d' Curves Across Channels: How would you interpret the effect size of this data?

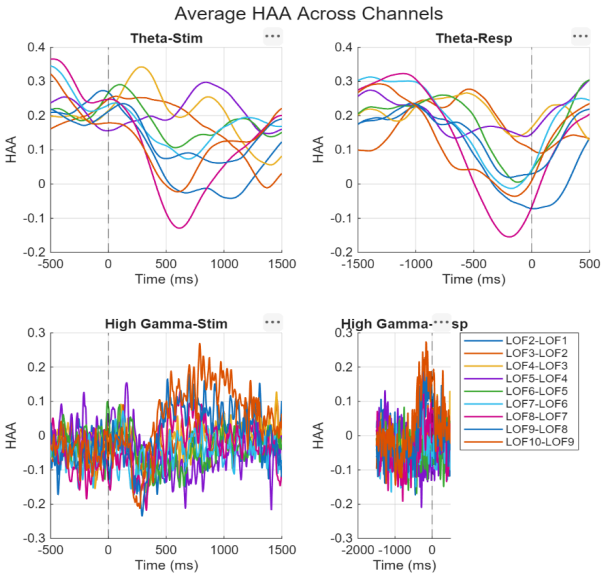
The effect size determines how well HAA separates low vs high conflict datapoints from our LFPs.

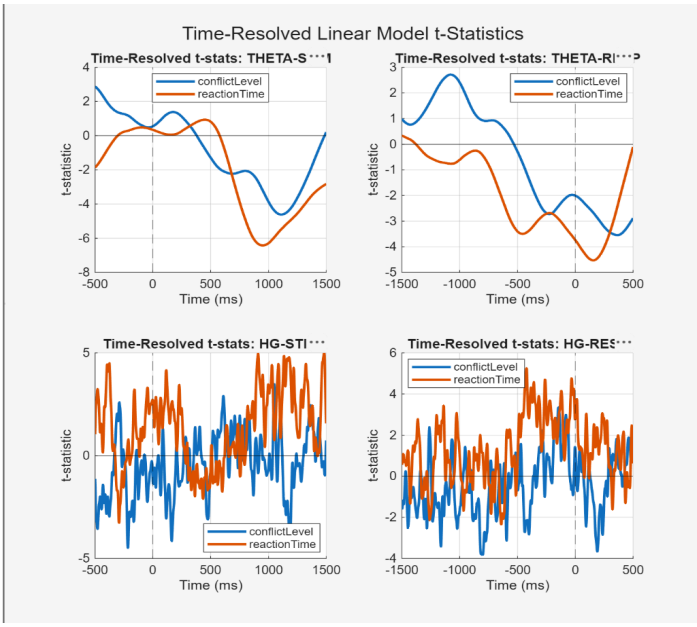
- Theta band stimulus aligned response: d'curve peaks at 0.5, thus it has a medium effect size. Because the peak is 1000ms after the stimulus onset, this tells us that theta activity is encoding conflict after the stimulus
- Theta band response aligned response: curves peak around 0.4 to 0.5, thus it has a medium effect size. The peaks are shortly after stimulus between -250 and 250 ms, thus theta tracks the decisions near the response
- High gamma stimulus aligned: Noisy data with a couple peaks in the 0.5 region. However there is no global maximum that is visually different from other time points, therefore the effect size is weak
- High gamma response aligned: similar to the stimulus aligned graph, there are scattered spikes everywhere. Channel 2 has a distinct spike at 0.7 which could indicate a large effect size for that specific channel, however the other channels do not exhibit the same effect size.

PART 2:

For Figure 6 Time-Resolved Linear Model t-Statistics: Plot the time course of the t-statistics for each fixed effect in each group-level model. What do you notice? Do these results differ from the channel-level results?

The time course of all the 4 subplots follow similar looking behavior to Figure 1 which encompasses all the channels as its own separate entity. We see that in the window, high Gamma waves have significantly more activity, with the t-statistic for reactionTime fluctuating at a period of approximately 1000ms. This tells us that in the gamma frequency there is much more activity than in the theta band that is involved with rapid decision making in response to the stimulus. On the other hand, for the theta band,we see a negative t-statistic around the stimulus for both the stimulus and response aligned sub plots. A negative t-statistic typically implies a negative correlation of power decrease in that frequency band. Thus, theta bands do not seem to be involved with reacting to a stimulus onset.





Plot the FDR adjusted p-values over our t-statistic.

I chose to overlay the FDR corrected time resolved effect for the theta band stimulus aligned t-statistic over time graph. The FDR significant overlay highlighted the extreme negative peak at 1000ms. This may indicate that the negative significance of the theta wave in response to the stimulus may be in the wrong direction, and lower HAA has more conflict level impact than what the graph implies.

High Gamma signals look noisier than the Theta signals. Name one way to rectify this.

Because there is a peak around the stimulus onset in the t-statistic, we know that not all the oscillations are due to noise and there is significant activity in the high gamma frequency band. Some of the noise may be from muscle artifacts or electrode artifacts. We could use L2 regularization or ICA to single out relevant bands that have a t-statistic of greater than 0. Literature sources such as (*High Gamma Power Is Phase-Locked to Theta Oscillations in*

Human Neocortex - PMC) show that the gamma power is modulated by theta oscillations, thus an analog filter may introduce more artifacts than what we want. I could also use the Hilbert Transform.

Table of Contents

..... 1
PART 1: Channel-Level Encoding Effect Sizes 1
Average HAA plots for stimulus-aligned and response aligned HAA activity 2
Single bipolar channel average response channel histogram 3
Calculate AUC for each bipolar channel at each time point 4
Plot AUC Curves 5
Calculate d-prime 6
Plot d-primes 6
PART 2: GROUP-LEVEL ENCODING USING LINEAR MODELS 7
Time-resolved linear modeling using fitlm 10
Plot time resolved t-statistics for each time point 12
Inspect a band and event to run a statistical test using False Discovery Rate (FDR) correction 13


```
close all; clear; clc;
load('HW4Data.mat');
```

PART 1: Channel-Level Encoding Effect Sizes

Extract variables

```
% Time vectors
t.stim = erpParametersStimulus.timeWindow;
t.resp = erpParametersResponse.timeWindow;

evResp=eventTableResponse; % Event Table Response

% Labels for AUC
labels.stim = eventTableStimulus.conflictLevel(:);
labels.resp = evResp.conflictLevel(:);

% Channel info
channelLabels = bipolarContactInfo.bipolarLabel;
numChannels   = size(thetaHaa, 2);

% Band/alignment names
bands = {'theta','hg'};
aligns = {'stim','resp'};

% Organize HAA data
data.theta_stim = thetaHaa(1,:);
data.theta_resp = thetaHaa(2,:);
data.hg_stim = highGammaHaa(1,:);
data.hg_resp = highGammaHaa(2,:);
```

Average HAA plots for stimulus-aligned and response aligned HAA activity

```
figure;
plotTitles = {'Theta-Stim','Theta-Resp','High Gamma-Stim','High Gamma-Resp'};
idx = 1;

for b = 1:2
    for a = 1:2
        subplot(2,2,idx); hold on;

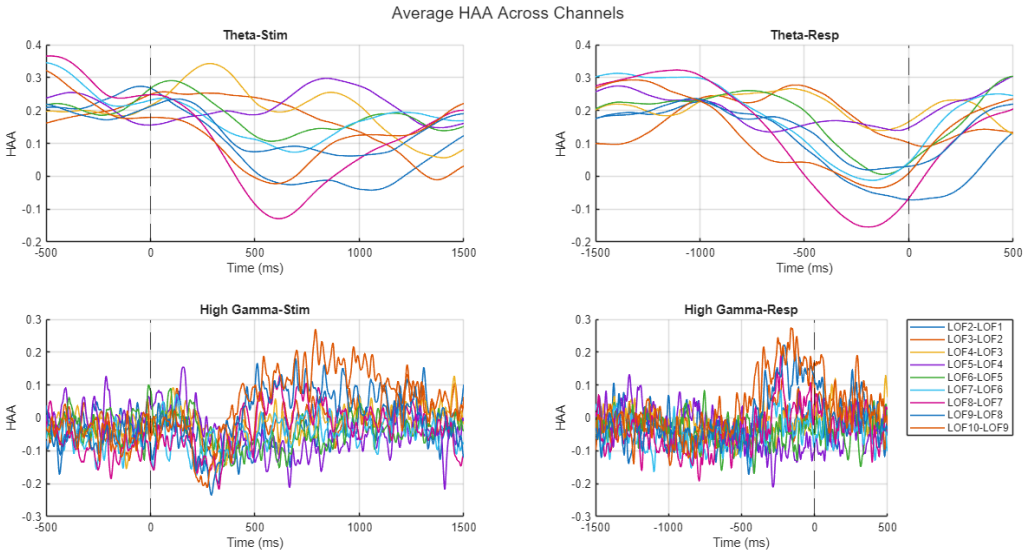
        key = sprintf('%s_%s', bands{b}, aligns{a});
        tvec = t.(aligns{a});

        for ch = 1:numChannels
            X = data.(key){ch};    % trials * time
            plot(tvec, mean(X,1,'omitnan'), 'LineWidth', 1.2);
        end

        xline(0,'k--');
        grid on;
        xlabel('Time (ms)');
        ylabel('HAA');
        title(plotTitles{idx});

        idx = idx + 1;
    end
end

legend(channelLabels,'Location','bestoutside');
sgtitle('Average HAA Across Channels');
```



Single bipolar channel average response channel histogram

```
singleChannel = 2;
alignType = 'resp';
targetTime = -200; % ms

theta_single = data.(sprintf('theta_%s', alignType)){singleChannel};
hg_single    = data.(sprintf('hg_%s', alignType)){singleChannel};

tvec = t.(alignType);
[~, idxT] = min(abs(tvec - targetTime));

figure;

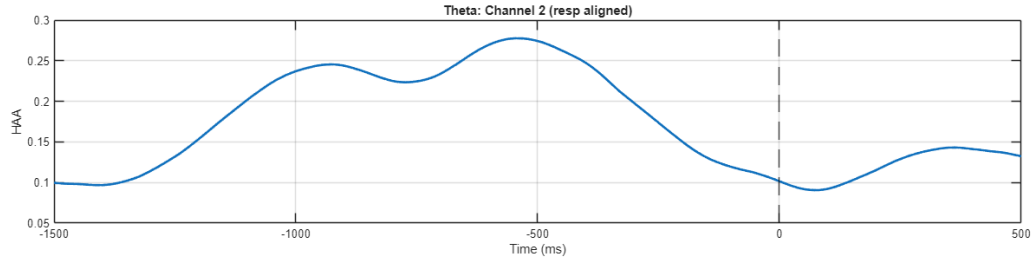
subplot(2,1,1);
plot(tvec, mean(theta_single,1,'omitnan'), 'LineWidth', 2);
xline(0,'k--');
grid on;
xlabel('Time (ms)');
ylabel('HAA');
title(sprintf('Theta: Channel %d (%s aligned)', singleChannel, alignType));

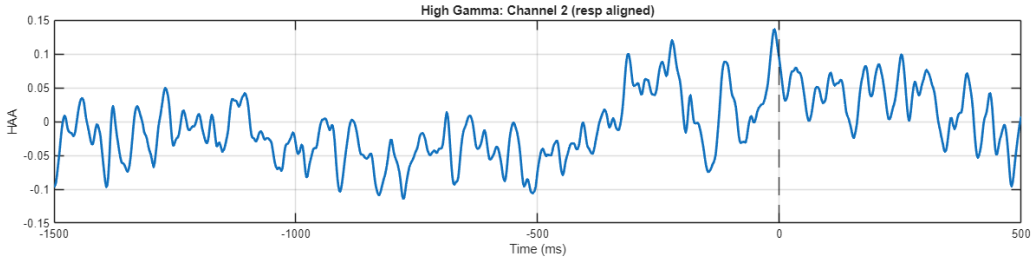
subplot(2,1,2);
plot(tvec, mean(hg_single,1,'omitnan'), 'LineWidth', 2);
xline(0,'k--');
grid on;
xlabel('Time (ms)');
ylabel('HAA');
title(sprintf('High Gamma: Channel %d (%s aligned)', singleChannel,
alignType));

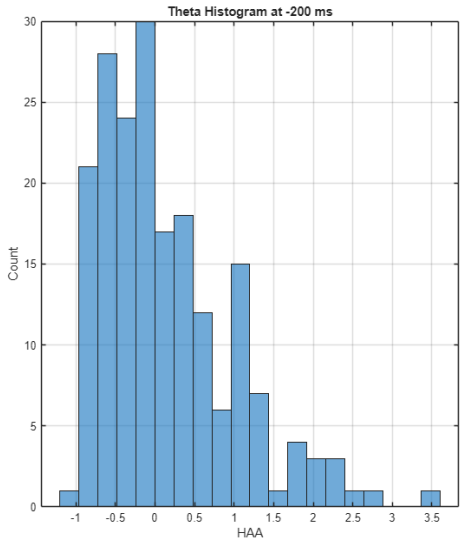
figure;

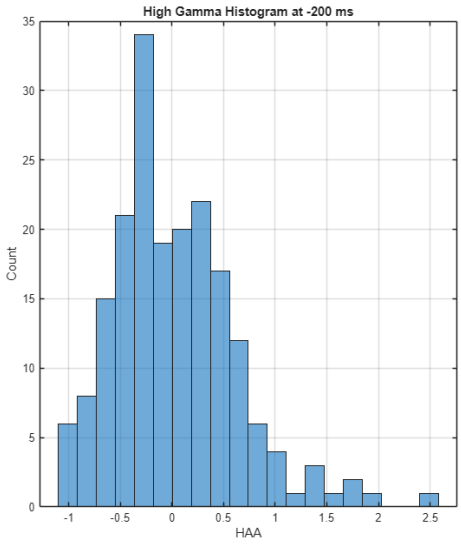
subplot(1,2,1);
histogram(theta_single(:,idxT), 20);
grid on;
xlabel('HAA');
ylabel('Count');
title(sprintf('Theta Histogram at %d ms', tvec(idxT)));

subplot(1,2,2);
histogram(hg_single(:,idxT), 20);
grid on;
xlabel('HAA');
ylabel('Count');
title(sprintf('High Gamma Histogram at %d ms', tvec(idxT)));
```









Calculate AUC for each bipolar channel at each time point

```
AUC = struct();

for b = 1:2
    for a = 1:2
        key  = sprintf('%s_%s', bands{b}, aligns{a});
        lbl  = labels.(aligns{a});
        tvec = t.(aligns{a});

        A = nan(length(tvec), numChannels);
```

```
        for ch = 1:numChannels
            X = data.(key){ch};    % trials * time
            A(:,ch) = computeAUCvectorized(X, lbl)';
        end

        AUC.(key) = A;
    end
end
```

Plot AUC Curves

```
figure;
idx = 1;

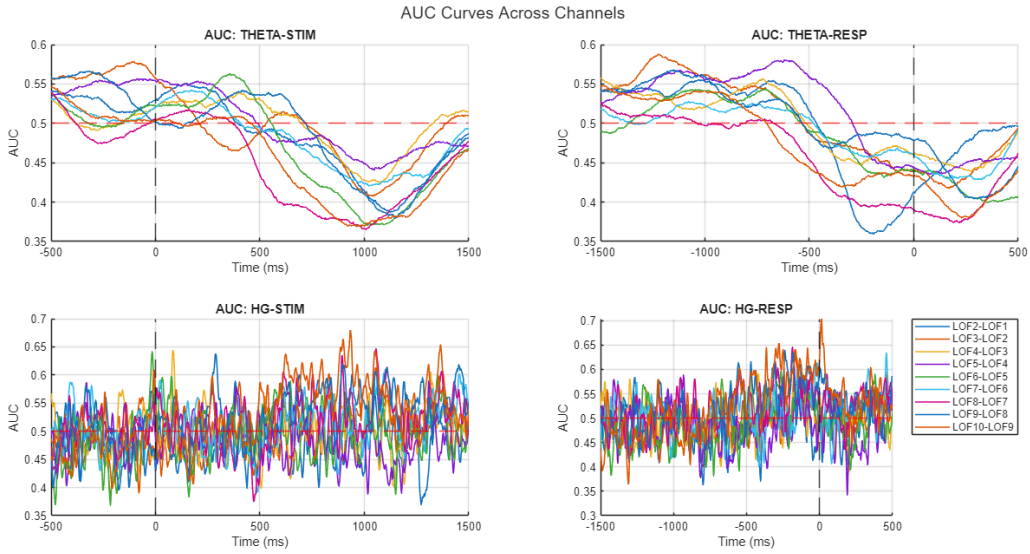
for b = 1:2
    for a = 1:2
        key  = sprintf('%s_%s', bands{b}, aligns{a});
        tvec = t.(aligns{a});

        subplot(2,2,idx); hold on;
        plot(tvec, AUC.(key), 'LineWidth', 1.2);
        xline(0,'k--');
        yline(0.5,'r--');
        grid on;

        xlabel('Time (ms)');
        ylabel('AUC');
        title(sprintf('AUC: %s', upper(strrep(key,'_','-'))));

        idx = idx + 1;
    end
end

legend(channelLabels,'Location','bestoutside');
sgtitle('AUC Curves Across Channels');
```



Calculate d-prime

```
dPrime = struct();
fn = fieldnames(AUC);

for i = 1:numel(fn)
    A = AUC.(fn{i});

    % Reflect values < 0.5
    A(A < 0.5) = 1 - A(A < 0.5);

    % Avoid Inf from norminv(0) or norminv(1)
    epsVal = 1e-6;
    A(A <= 0) = epsVal;
    A(A >= 1) = 1 - epsVal;

    dPrime.(fn{i}) = sqrt(2) * norminv(A);
end
```

Plot d-primes

```
figure;
idx = 1;

for b = 1:2
    for a = 1:2
        key  = sprintf('%s_%s', bands{b}, aligns{a});
        tvec = t.(aligns{a});

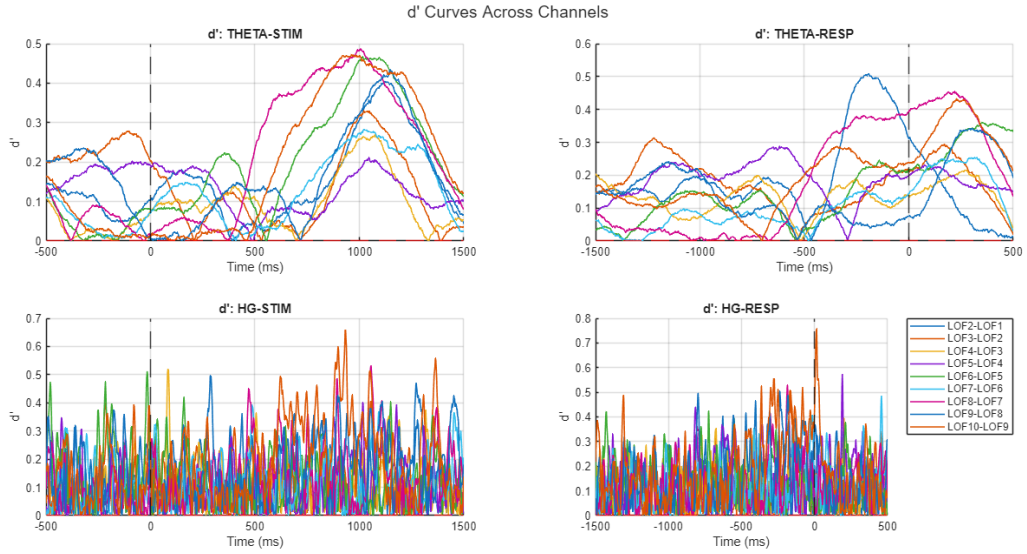
        subplot(2,2,idx); hold on;
        plot(tvec, dPrime.(key), 'LineWidth', 1.2);
        xline(0,'k--');
        yline(0,'r--');
```

```
        grid on;

        xlabel('Time (ms)');
        ylabel('d''');
        title(sprintf('d'': %s', upper(strrep(key,'_','-'))));

        idx = idx + 1;
    end
end

legend(channelLabels,'Location','bestoutside');
sgtitle('d'' Curves Across Channels');
```



PART 2: GROUP-LEVEL ENCODING USING LINEAR MODELS

Wilkinson's Notation: HAA ~ conflictLevel + reactionTime

```
disp('--- Building combined tables and fitting pooled linear models ---');

combined = struct();
models   = struct();

for b = 1:2
    for a = 1:2

        key = sprintf('%s_%s', bands{b}, aligns{a});
        disp(['Processing pooled model: ' key]);

        % Select event table
        if strcmp(aligns{a}, 'stim')
            ev = eventTableStimulus;
        else
```

```
            ev = evResp;
        end

        allHAA = [];
        allConflict = [];
        allRT = [];
        allChan = [];

        for ch = 1:numChannels
            X = data.(key){ch}; % trials * time
            [nTrials, nTime] = size(X);

            haaVec = X(:);

            conflict = repmat(ev.conflictLevel(:), nTime, 1);
            rt = repmat(ev.reactionTime(:), nTime, 1);

            allHAA = [allHAA; haaVec];
            allConflict = [allConflict; conflict];
            allRT = [allRT; rt];
            allChan = [allChan; repmat(ch, numel(haaVec), 1)];
        end

        T = table( ...
            allHAA, ...
            allConflict, ...
            allRT, ...
            categorical(allChan), ...
            'VariableNames',
{'HAA','conflictLevel','reactionTime','channel'});

        T = rmmissing(T);

        combined.(key) = T;
        models.(key)   = fitlm(T, 'HAA ~ conflictLevel + reactionTime +
channel');

        disp(models.(key));
    end
end

disp('--- Pooled linear modeling complete ---');
```

```
--- Building combined tables and fitting pooled linear models ---
Processing pooled model: theta_stim

Linear regression model:
    HAA ~ 1 + conflictLevel + reactionTime + channel

Estimated Coefficients:
                    Estimate        SE         tStat       pValue
                    ________    _________    _______    ___________

    (Intercept)       0.2919    0.0034866     83.721              0
```

conflictLevel	-0.034444	0.0015704	-21.934	1.2869e-106
reactionTime	-0.21067	0.0036884	-57.118	0
channel_2	0.073951	0.0029156	25.364	6.6484e-142
channel_3	0.10546	0.0029156	36.173	2.0878e-286
channel_4	0.11351	0.0029156	38.931	0
channel_5	0.087527	0.0029156	30.021	5.9471e-198
channel_6	0.08474	0.0029156	29.064	1.1217e-185
channel_7	0.02506	0.0029156	8.5951	8.3265e-18
channel_8	0.042297	0.0029156	14.507	1.0961e-47
channel_9	0.034635	0.0029156	11.879	1.5228e-32

Number of observations: 1738737, Error degrees of freedom: 1738726
Root Mean Squared Error: 0.906
R-squared: 0.00534, Adjusted R-Squared: 0.00533
F-statistic vs. constant model: 933, p-value = 0
Processing pooled model: theta_resp

Linear regression model:
    HAA ~ 1 + conflictLevel + reactionTime + channel

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.2776	0.0035003	79.309	0
conflictLevel	-0.018538	0.0015766	-11.758	6.4084e-32
reactionTime	-0.21479	0.0037028	-58.007	0
channel_2	0.085506	0.002927	29.213	1.4834e-187
channel_3	0.1213	0.002927	41.441	0
channel_4	0.11318	0.002927	38.668	0
channel_5	0.088929	0.002927	30.382	1.0651e-202
channel_6	0.093256	0.002927	31.86	1.0933e-222
channel_7	0.033584	0.002927	11.474	1.7926e-30
channel_8	0.054542	0.002927	18.634	1.7298e-77
channel_9	0.044611	0.002927	15.241	1.9001e-52

Number of observations: 1738737, Error degrees of freedom: 1738726
Root Mean Squared Error: 0.91
R-squared: 0.00481, Adjusted R-Squared: 0.00481
F-statistic vs. constant model: 840, p-value = 0
Processing pooled model: hg_stim

Linear regression model:
    HAA ~ 1 + conflictLevel + reactionTime + channel

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	-0.11713	0.0024885	-47.069	0
conflictLevel	-0.015425	0.0011209	-13.762	4.33e-43
reactionTime	0.11615	0.0026325	44.12	0

```
    channel_2         0.019111     0.002081      9.1837     4.1705e-20
    channel_3         0.011504     0.002081      5.5282     3.2355e-08
    channel_4        -0.012079     0.002081     -5.8048     6.4462e-09
    channel_5        -0.013054     0.002081     -6.2729     3.5454e-10
    channel_6        -0.024536     0.002081     -11.791     4.3612e-32
    channel_7        -0.011957     0.002081     -5.7459     9.1445e-09
    channel_8         0.023596     0.002081      11.339     8.4498e-30
    channel_9         0.062123     0.002081      29.853    8.9329e-196


Number of observations: 1738737, Error degrees of freedom: 1738726
Root Mean Squared Error: 0.647
R-squared: 0.00265,  Adjusted R-Squared: 0.00265
F-statistic vs. constant model: 463, p-value = 0
Processing pooled model: hg_resp

Linear regression model:
    HAA ~ 1 + conflictLevel + reactionTime + channel

Estimated Coefficients:
                       Estimate        SE         tStat       pValue
                      __________    _________    _______    ___________

    (Intercept)        -0.10896     0.0024922    -43.721             0
    conflictLevel     -0.014961     0.0011225    -13.328     1.5897e-40
    reactionTime        0.10755     0.0026364     40.794             0
    channel_2          0.019542      0.002084     9.3768     6.8095e-21
    channel_3          0.013484      0.002084     6.4703     9.7853e-11
    channel_4        -0.0091561      0.002084    -4.3934     1.1157e-05
    channel_5         -0.012726      0.002084    -6.1064     1.0193e-09
    channel_6         -0.023162      0.002084    -11.114     1.0738e-28
    channel_7         -0.010083      0.002084    -4.8383     1.3097e-06
    channel_8          0.026165      0.002084     12.555      3.742e-36
    channel_9          0.063386      0.002084     30.415    3.9134e-203


Number of observations: 1738737, Error degrees of freedom: 1738726
Root Mean Squared Error: 0.648
R-squared: 0.00248,  Adjusted R-Squared: 0.00247
F-statistic vs. constant model: 432, p-value = 0
--- Pooled linear modeling complete ---
```

Time-resolved linear modeling using fitlm

```
disp('--- Running time-resolved linear models ---');

timeResolvedStats = struct();

for b = 1:2
    for a = 1:2

        key = sprintf('%s_%s', bands{b}, aligns{a});
        disp(['Processing time-resolved model: ' key]);
```

```
        tvec = t.(aligns{a});
        nT = length(tvec);

        if strcmp(aligns{a}, 'stim')
            ev = eventTableStimulus;
        else
            ev = evResp;
        end

        t_conflict = nan(nT,1);
        p_conflict = nan(nT,1);
        t_rt= nan(nT,1);
        p_rt= nan(nT,1);

        for ti = 1:nT

            allHAA = [];
            allConflict = [];
            allRT = [];
            allChan = [];

            for ch = 1:numChannels
                X = data.(key){ch};   % trials x time

                if ti > size(X,2)
                    continue
                end

                y = X(:,ti);

                allHAA  = [allHAA; y];
                allConflict = [allConflict; ev.conflictLevel(:)]; % Conflict
Level
                allRT = [allRT; ev.reactionTime(:)]; % Reaction Time
                allChan  = [allChan; repmat(ch, length(y), 1)];
            end

            Tti = table( ...
                allHAA, ...
                allConflict, ...
                allRT, ...
                categorical(allChan), ...
                'VariableNames',
{'HAA','conflictLevel','reactionTime','channel'});

            Tti = rmmissing(Tti);

            if height(Tti) < 10
                continue
            end

            if numel(unique(Tti.conflictLevel)) < 2
                continue
```

```
            end

            if numel(unique(Tti.reactionTime)) < 2
                continue
            end

            mdl = fitlm(Tti, 'HAA ~ conflictLevel + reactionTime + channel');
            coef = mdl.Coefficients;

            rowNames = coef.Properties.RowNames;

            if ismember('conflictLevel', rowNames)
                t_conflict(ti) = coef{'conflictLevel','tStat'};
                p_conflict(ti) = coef{'conflictLevel','pValue'};
            end

            if ismember('reactionTime', rowNames)
                t_rt(ti) = coef{'reactionTime','tStat'};
                p_rt(ti) = coef{'reactionTime','pValue'};
            end
        end
        % Extract statistical significance values from time resolved data
        timeResolvedStats.(key).time  = tvec;
        timeResolvedStats.(key).t_conflict = t_conflict;
        timeResolvedStats.(key).p_conflict = p_conflict;
        timeResolvedStats.(key).t_rt = t_rt;
        timeResolvedStats.(key).p_rt = p_rt;
    end
end

disp('--- Time-resolved linear modeling complete ---');
```

```
--- Running time-resolved linear models ---
Processing time-resolved model: theta_stim
Processing time-resolved model: theta_resp
Processing time-resolved model: hg_stim
Processing time-resolved model: hg_resp
--- Time-resolved linear modeling complete ---
```

Plot time resolved t-statistics for each time point

```
figure;
plotIdx = 1;

for b = 1:2
    for a = 1:2

        key   = sprintf('%s_%s', bands{b}, aligns{a});
        stats = timeResolvedStats.(key);

        subplot(2,2,plotIdx); hold on;
```

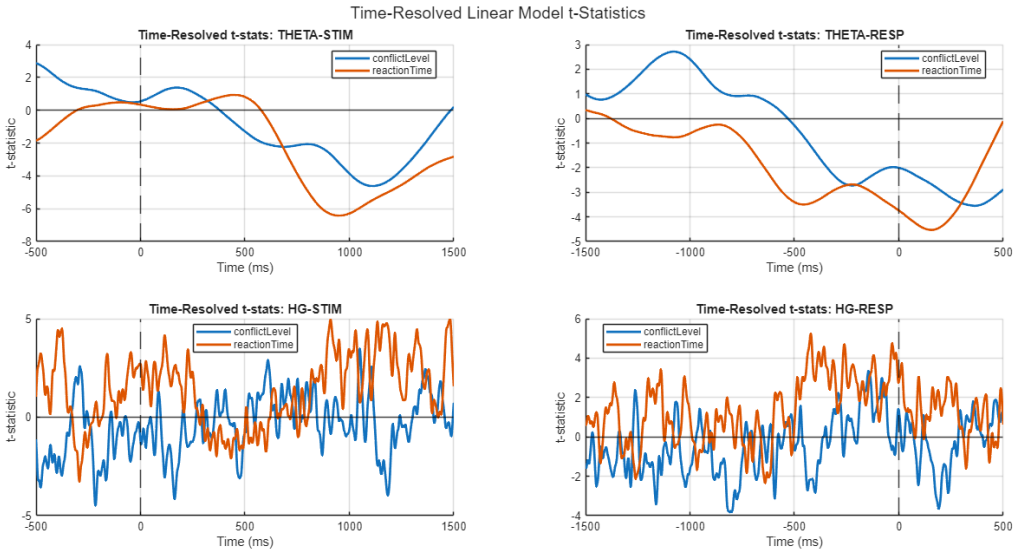
```
        plot(stats.time, stats.t_conflict, 'LineWidth', 2);
        plot(stats.time, stats.t_rt, 'LineWidth', 2);

        xline(0,'k--');
        yline(0,'k-');
        grid on;

        xlabel('Time (ms)');
        ylabel('t-statistic');
        title(sprintf('Time-Resolved t-stats: %s',
upper(strrep(key,'_','-'))));
        legend({'conflictLevel','reactionTime'}, 'Location','best');

        plotIdx = plotIdx + 1;
    end
end

sgtitle('Time-Resolved Linear Model t-Statistics');
```



Inspect a band and event to run a statistical test using False Discovery Rate (FDR) correction

Uses fdr_bh.m

```
band = 'theta'; % 'theta' or 'hg'
align = 'stim'; % 'stim' or 'resp'
predictor = 'conflict'; % 'conflict' or 'rt'

key = sprintf('%s_%s', band, align); %extract the correct band and aligned
response
```

```
stats = timeResolvedStats.(key);
tvec = stats.time; % time window

switch predictor % extract the correct table values
    % by using the switch classifier it makes it easier to switch between
    % variables for my own curiosity and also for debugging purposes so i
    % don't have to rewrite multiple chunks of code
    case 'conflict'
        pvals = stats.p_conflict;
        tvals = stats.t_conflict;
        predLabel = 'Conflict Level';
    case 'rt'
        pvals = stats.p_rt;
        tvals  = stats.t_rt;
        predLabel = 'Reaction Time';
    otherwise
        error('predictor must be ''conflict'' or ''rt''.');
end

validIdx = ~isnan(pvals) & ~isnan(tvals);
p_clean  = pvals(validIdx);

if isempty(p_clean)
    warning('No valid p-values available for FDR correction.');
    h_full = false(size(pvals));
else
    [h, crit_p, adj_ci_cvrg, adj_p] = fdr_bh(p_clean, 0.05, 'pdep', 'no');
    h_full = false(size(pvals));
    h_full(validIdx) = h;
end

figure; hold on;

plot(tvec, tvals, 'LineWidth', 2);
xline(0,'k--');
yline(0,'k-');
grid on;

sigTimes = tvec(h_full);
sigTvals = tvals(h_full);

scatter(sigTimes, sigTvals, 40, 'r', 'filled');

xlabel('Time (ms)');
ylabel('t-statistic');
title(sprintf('FDR-Corrected Time-Resolved Effect: %s (%s-%s)', ...
    predLabel, upper(band), upper(align)));
legend({'t-statistic','Event onset','Zero line','FDR-significant'}, ...
    'Location','best');

% computers AUC
function aucVec = computeAUCvectorized(X, labels)
% X : trials * time
% labels : trials * 1
```

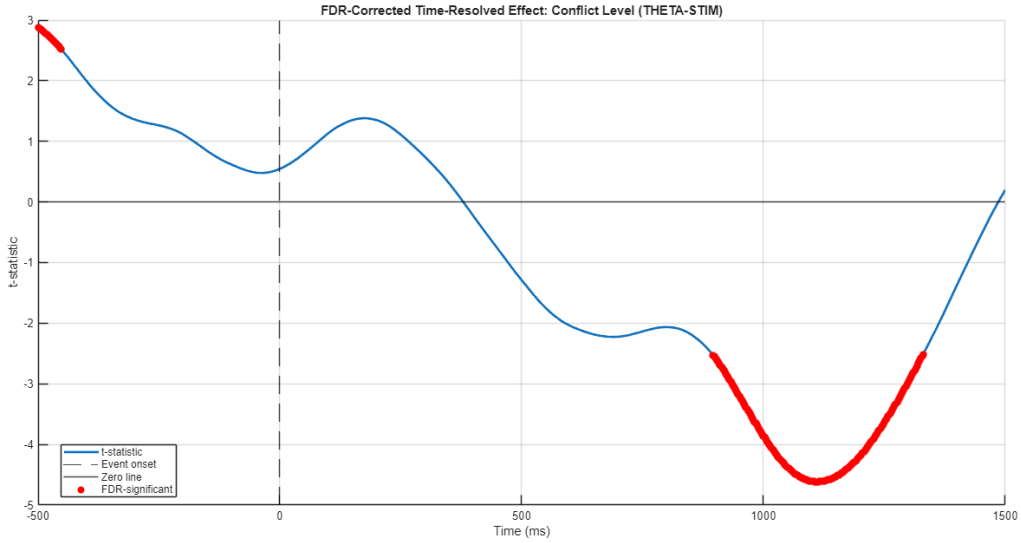
```
% Returns:
% aucVec : 1 *time

    nT = size(X,2);
    aucVec = nan(1,nT);

    for ti = 1:nT
        valid = ~isnan(labels) & ~isnan(X(:,ti));
        x = X(valid,ti);
        y = labels(valid);

        if numel(unique(y)) < 2
            continue
        end

        [~,~,~,auc] = perfcurve(y, x, 1);
        aucVec(ti) = auc;
    end
end
```



Published with MATLAB® R2026a